

revert Change

放弃本地未推送的改动并回到上一次 push 的状态，主要取决于你目前的改动是否已经进行了 `commit`（提交）。

注意：以下所有操作都会永久删除你目前的本地改动，且无法恢复。执行前请确认你确实不再需要这些修改。

你可以根据自己的情况选择以下两种方法之一：

情况一：pushed，没有 `commit`（未提交） (直接删掉上一个git push的存档)

如果你只是修改了文件，还没有执行 `git commit`，你可以直接用这两条命令来清空工作区：

1. 撤销所有已追踪（Tracked）文件的修改：

```
Bash
```

```
git reset --hard HEAD
```

2. 删除所有未追踪（Untracked）的新文件和文件夹：

```
Bash
```

```
git clean -fd
```

(比如你新导入的贴图或新建的蓝图，如果没有被 `git` 追踪过，`reset` 是管不到它们的，必须用 `clean` 来清理。)

在敲那句让你害怕的命令之前，你可以先用这条完全无害的命令来检查 `HEAD` 到底指在哪里：

```
Bash
```

```
git log -1
```

这条命令的意思是“只显示当前所在的这 1 个 commit”。根据你之前的记录，敲完后你一定会看到类似这样的输出：

```
commit cd9d14706 (HEAD -> main) ... sprint1
```

只要看到你看到括号里写着 (HEAD -> main)，并且下面写着你的 commit 名字是 `sprint1`，这就铁证如山地说明：你的 HEAD 牢牢钉在 `sprint1` 上。

确认完这个，你就可以放心大胆地执行 `git reset --hard HEAD` 了。

不确定就备份

情况二：改动已经 commit，但没有 push（已提交）

如果你已经把改动 commit 到了本地历史记录中，你需要将本地分支的指针强制退回到远端仓库的状态：

1. 获取远端的最新状态（确保你和远端同步）：

```
Bash
```

```
git fetch origin
```

2. 将本地状态强制重置为远端分支的状态：

```
Bash
```

```
git reset --hard origin/你的分支名
```

(如果你在 `main` 分支，就把 你的分支名 替换为 `main`，如 `git reset --hard origin/main`)

针对 Git LFS 的额外步骤

因为你使用了 Git LFS（这在处理 3D 资产、Photoshop 源文件或动画文件时很常见），在使用 `--hard` 重置之后，有时本地工作区可能会把 LFS 追踪的大文件变成只有几 KB 的“指针文件”，导致资产在引擎里无法打开或显示损坏。

如果在 `reset` 之后遇到了文件读取错误，请运行以下命令来强制拉取真实的物理文件：

```
Bash
```

```
git lfs checkout
```

或者：

```
Bash
```

```
git lfs pull
```